

JAVA NOTES

Revision of Class IX Syllabus

Programming paradigms are a way to classify programming languages based on their features. Paradigm means organizing principle of a program. It is an approach to programming.

Procedure Oriented Programming: A Procedure oriented programming approach allows the users to develop their logic by using a number of functions that would enhance the program's productivity.

Example BASIC, COBOL, C

Object Oriented Programming: An Object Oriented Programming is a modular approach, which allows the data to be applied with a stipulated program area. It also provides the reusability feature to develop productive logic, which means to give more emphasis on data.

Basic Principles of OOP:

1. **Abstraction**: The act of representing essential features, without including the background details.
2. **Inheritance**: Capability of one class of things to inherit capabilities or properties from another class.
3. **Encapsulation**: Wrapping up of data and functions into a single unit.
4. **Polymorphism**: Polymorphism is the ability for a message or data to be processed in more than one form.

Java compilation process:

1. Java programs are written in “.java” file. (source code) and then compiled by Java compiler.
2. **Byte code**: Java compiler converts the source code into an intermediate binary form called the byte code.
3. **Java Virtual Machine (JVM)**: It a java interpreter that converts byte code into machine to various platforms.
4. **Just In Time(JIT)**: It is part of the JVM and it compiles byte code into executable code in real time, one piece-by-piece, demand basis

Characteristics of Java:

1. Write Once Run Anywhere (WORA): The Java programs need to be written just once, which can run on different platforms without making changes in the Java program.
2. Light Weight Code: With Java, no huge coding is required.
3. Security: Java offers many enhanced security features.
4. Object Oriented Language: Java is Object Oriented language, thereby, very near to real world.
5. Platform Independent: Java is essentially platform independent. Change of platform does not affect the original Java program.

Types of Java program:

1. Internet Applets: The programs executed inside the Java based web browser.
2. Java Applications: The programs developed by the users.

Java libraries:

A package is a collection of various classes. Each class contains different functions.

A package can be included in the program by using a keyword 'import'.

ex) import java.io.*; import

java.util.*;

The (*) sign denotes that all the classes of the concerning package will be made available for use in your program.

Keywords or Reserved words:

Java reserved words or the keywords are the words which carry special meaning to the system compiler. Such words cannot be used for naming a variable in the program.

case , switch, else, break , static, do, const, throws, float , char, try, int, double, void, goto, for, while, new, import , boolean, long, if, byte , package, private, catch, short, public, class, default.

Output statement: System.out.println();

Comment line:

The comment statement is a non executable statement in Java.

These are the remarks given by the user.

Types: single line comment	//-----
Multiline comment	/* ----- */
Document comment	/** ----- **/

ASCII

A - Z 65 - 90

a-z 97 - 122

0-9 48 - 57

white space 32

Token: The smallest individual unit in a program is known as a Token

Types:

1. **Keywords:** Keywords are the reserved words that convey a special meaning to the language compiler. These are reserved for special purpose.
Example) class, int, void, float
2. **Literals:** Literals or constants are data items that are fixed data values (do not during the execution of the program). **Types:**
 - a) Integer Literals are whole numbers without any fractional part.
Decimal, Octal, Hexa decimal.
Ex) a= 505, b=-15
 - b) Real literals are numbers having fractional parts.
Ex) p=16.79 , q=-1.005
 - c) character literal is one character enclosed in single quotes.
Ex) 'x', '9', '*'
 - d) String literals are multiple character enclosed with double quotes.
Ex) name= "aravind"

e) Boolean literal: the Boolean type has two values, true or false.

f) Null literal has one value, the null reference. A null literal is always of the null type.

3. **Separators:** The following nine ASCII characters are the separators(punctuators).

() { } [] ; , .

4. **Identifier:** Identifiers are fundamental building block of a program such as a variable, class, method etc.

It's used as the general terminology for the names given to different parts of the program. `double area=15.6;`

Rules for forming identifiers:

- Identifiers can have alphabets, digits and underscore and dollar sign characters.
 - They must not be a keyword or Boolean literal or null literal
 - They must not begin with a digit
 - They can be of any length
5. **Operators:** Operators are special symbols that cause an action to take place.

Arithmetic, relational, logical, conditional operators. `A=b*c;`

Escape sequences: Nongraphic characters are those characters that cannot be typed directly from keyboard.

E.g. backspace, tabs etc.,

An escape sequence is represented by a backslash (\) followed by one or more characters

Escape sequence	Nongraphic character
<code>\b</code>	Backspace
<code>\f</code>	Form feed
<code>\n</code>	Newline or linefeed
<code>\r</code>	Carriage return
<code>\t</code>	Tab space
<code>\\</code>	Backslash
<code>\'</code>	Single quotes
<code>\"</code>	Double quotes
<code>\?</code>	Question mark
<code>\0</code>	null

Data types:

Data Types are means to identify the type of data and associated operations of handling it.

Two types:

1. Primitive or fundamental or Intrinsic (predefined)

int, long, float, double, char, short, byte , boolean

2. Reference or composite (user defined) class,
string, array

PRIMITIVE DATA TYPES

NOTE: (1byte=8bits)

Primitive Data Type	Size	Details
byte	1 byte	Stores positive and negative numbers ranging from -128 to 127 .
int	4 bytes	Stores positive and negative numbers ranging from -2,147,483,648 to 2,147,483,647 .
short	2 bytes	Stores positive and negative numbers ranging from -32,768 to 32,767 .
long	8 bytes	Stores positive and negative numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 .
float	4 bytes	Stores Decimal numbers. It can be used for storing numbers having 6 to 7 decimal digits
double	8 bytes	Stores Decimal numbers. It can be used for storing numbers having 15 decimal digits .
boolean	1 bit	Can Store Only true or false .
char	2 bytes	It can be used for storing only a single character , letter or ASCII values.

Variable:

A variable is a named temporary memory location to store the values.

eg) float b = 15.5; <data type> <var name> =

<value>;

char ch = 'G';

Initializing a variable:

Initializing a variable means specifying an initial value to assign to it (i.e., before it is used at all). Notice that a **variable** that is not **initialized** does not have a defined value, hence it cannot be used until it is assigned such a value.

Static initialization: Initialize the variable at the time of declaration int b=5,a=0;

float c=0.0

Data type	Value
Int	0
Long	0
Float	0.0
Double	0.0
Char	'\u0000'
String	“ ”
Boolean	False

Dynamic initialization : Initialize the variable at program execution time.(at run time)

```
int b=5, c=6; int A= b+c;
```

Arithmetic expression:

A set of variables, constants and arithmetical operators used together to yield a meaningful result is known as Arithmetical Expression. $d = a*b + c/4$;

Types:

1. ***Pure Expression:*** An arithmetical expression that uses all its components of same data type.

```
int a,b;
```

```
int c=a+b*4;
```

2. ***Impure expression:*** An arithmetical expression in which one or more components are of different data types.

```
int a; float f; double d; double s= a*f/d;
```

Type conversion: In a mixed expression, the result can be obtained in any one form of its datatypes. Hence, it is needed to convert the various datatypes into a single type. Such conversion is termed as Type conversion. [Converting one form of data type into another form of data type]

Types:

1. *Implicit type conversion* (Coercion)

The data type of the result gets automatically converted into the highest data type available into the expression without any intervention of the user.

Hierarchy of Data types (DFL ISC B)

```
double -> float-> long-> int -> short -> char -> byte int a; float b; double c;
```

```
d= a+b*c;
```

The data type of d is double.

1. *Explicit conversion* (Type casting)

When the data type gets converted to another data type after the user's intervention.

```
int a; float b; double c; d= (float) a+b*c;
```

The data type of d is float.

Note: Explicit conversion of literals is allowed in Java programming.

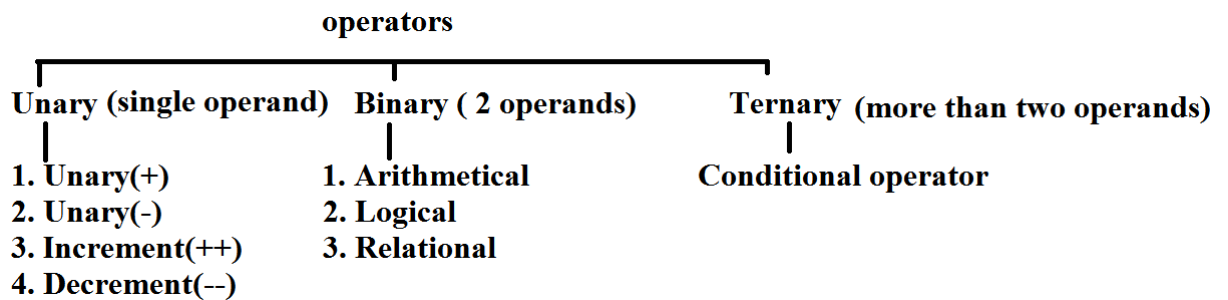
float a= 14.67 F; double

b= 14.67 D; int c= 59;

long f= 59 L;

Operators: Operators are special symbols that cause an action to take place.

Types:



Unary operators:

1. Unary(+): int a=5;
 int b= +a; //5
2. Unary(-): int a=5; int
 b= -a; //-5
 int a=-5; int b=-a ; //5
3. Increment operator(++): (increased by 1)
 a) Post increment a++
 b) Pre increment ++a
4. Decrement operator(--): (decreased by 1)
 a) Post decrement a--
 b) Pre decrement --a

Binary operators: (Two operands and one operator)

1. Arithmetic operators: + , - , * , / , % a=5, b=2
 a+b, a - b, a*b, a/b, a%b

2. Relational operators:(comparing the values)
 > , >= , < , <= , == , !=
 a>b, a>=b, a<b, a<=b, a==b, a!=b

3. Logical operators: AND (&&) , OR (||) , NOT (!) **Ternary**

operator (or) Conditional operator: (? :)

Syntax: Condition ? Expression 1 : Expression 2 ; Example;

1) a = 19 , b = 15

max = (a > b) ? a : b ;

2) salary = 15000

bonus = (salary > 10000) ? salary*15/100 : salary* 5/100 ;

Special operators:

new: **dynamically** allocate the memory for the object. Example:
example e = new example();

Dot (.) operator: invoking members of class Example:

System.out.println(); (System.in)

Operator precedence:

Operator precedence determines the order in which the operators in an expression are evaluated.

Object is an instance of a class. It is an identifiable entity with some characteristics and behavior.

Class represents a set of objects that share common characteristics and behavior.

Abstraction : it is the act of representing essential features and hiding the background details.

Encapsulation: it is an act of wrapping of the data and function as one single unit. Provides insulation of the data from the direct access by the program.

Inheritance : it is the capability of one class to inherit the properties from another class

Polymorphism: it is the ability of data to be processed in more than one form

Message passing – when the objects need to interact with one another, they pass /request information to one another. This interaction is known as message passing.

Compiler : It is a software which converts high level language program to machine language and viz. as a whole. It is faster but hard to debug.

Interpreter: It is a software which converts high level language program to machine language and viz line by line. It is slow but easy to debug.

Bytecode : When a java source code is compiled the resultant got is called a bytecode.

Keywords: These are words have a specific meaning to a language compiler.

Tokens: It is the smallest unit of a program

Literals : is a token whose value does not change during the execution of a program.

Comments: Giving remarks for a statement is called as a comment. There are 3 different types of giving a comment.

- // single line comment
- /*
..... */ multi line comment
- /** */ documentation

Jdk – Java development kit.

Constant: a data item that never changes its value during a program runs.

Final: The keyword final makes a variable as constant. i.e., whose value cannot be changed during the program execution

Operators: a symbol used to depict a mathematical or logical operation.

Arithmetic operators: they provide facilitation to the mathematical calculations within a program: they are + - * / %

Assignment operators: These operators are used to assign the value of an expression to a variable : =

Shorthand operators : allows ease while programming instructions and feature with all the available operators with two operands :

+= -= *= /= %=

Relational operators: provide facilitation for comparing numbers and characters for calculation and do not function with strings.: >

< <= >= == !=

Increment operator : is used to increment the variable by 1: ++

Logical operators: used to conduct the logical combination of Boolean values within a program.: && || !

Expression : is a combination of operators, constants and variables.

Type conversion: The process of converting one predefined type into another.

Explicit type conversion: The process of converting one predefined type into another.

It is user defined conversion that forces an expression to be of specific type (also called as **type casting**) e.g. `int x= 65; char c = (char) x;` it is called type casting.

Implicit type conversion: The process of converting one predefined type into another is called type conversion. It is performed invisible without user intervention and hence known as automatic conversion.

When two operands of different types are encountered in the same expression the lower type variable is converted to the type of the higher type variable automatically and is also known as **type promotion**.

byte x=8; int y=x; it is called **coercion**

Variables

A variable is defined as a location in the memory of the computer where the values are stored.

Static initialization: it is an expression that initializes a variable during compile time.

E.g., `int a=10;`

Dynamic initialization: it is an expression that initializes a variable during runtime;

E.g.: `int a= c * b;`

Package : is a collection of classes. Each package includes related built-in functions.

Statement : A set of instructions which terminate with a colon are called a statement.

E.g., int a =4;

Compound statement / Block statement: Multiline statements are called compound/block statement. They are enclosed in curly brackets { }

E.g., if(a>b)

```
{  
    a=4;  
    b=0;  
}
```

SELECTION STATEMENT: These are statements which allow to choose a set of instructions for execution depending upon an expression.

if-else statement tests an expression and depending upon its truth value one of the two sets of action is executed.

Dangling else: In Nested if statement the number of if is more than the number of else.(unmatched if and else)

Switch is a multiple branching statement, this statement tests the value of an expression against a list of integer or character constants for equality.

Default statement gets executed when no match is found in the switch cases.

Fall through: the fall of control to the following cases of matching case (or) execution of multiple cases after matching takes place in a switch statement.

Iteration statements : executing a set of statements repeatedly until a given condition is met.

Types: for, while and do- while statements.

for is the easiest to understand of the Java loops. All its loop control elements are gathered in one place(on the top of the loop)

While statement is another looping statement and it is entry controlled. The statement will work only if the condition is true.

Do-while: It is another looping statement and it is exit controlled. The statement will work once even if the condition is false.

JUMP STATEMENTS: unconditionally transfer program control within a function.

Break: it is used as a terminator from the enclosed loop and transfers the control to the statement after the loop.(terminate the current loop)

(or)

Break : When break statement is executed it comes out of the inner most loop(if it is a loop statement) and it comes out of a switch-case statement.

continue: Causes the control to transfer from within the block to the next iteration of the loop. (Terminates the current iteration)

POP and OOPs

POP	OOP
Emphasis is given to functions	Emphasis is given to functions and data

= & ==

=	==
= is an assignment operator, used to assign values to a variable	== is a relational operator, used to check for equality
e.g., a=4;	if(a == 5)

Variables & constant

Variable	Constant
A variable is defined as a location in the memory of the computer where the value is stored.	A constant is the value which is stored in a variable
E.g.: a, amount	“Ida” “37 P.S.M. st”

Pre increment and post increment both increment the value by 1.

Pre increment	Post increment
Prefix the variable gets incremented first and then use.	post increment is incremented after the operation of function of its associate operator,
Whereas e.g., a=4; a+=++a; =9	a+=a++; = 8

& and &&

&	&&
& is a bitwise operator (for the manipulation of data at the bit level)	&& is a logical operator(the logical operator evaluates to true value if either of the 2 expressions is true.

Implicit Type conversion & Explicit type conversion

Implicit	Explicit
Is a conversion of one data type into another by the compiler without the user's intervention	It is user defined that forces an expression to be of specific type
Implicit –E.g. float a=4.5f; double x=987.9998;	int x=66; char c=(char)x;
If any operation is done using the above data types then the resultant will be double.	

Expression and operators

Expression	Operators
Expression is a combination of operators, constants and variables. It can be arithmetic, relational etc. E.g. : $u+1/2$ at^2	Operators : it is a symbol used to represent logical or arithmetic operations E.g. : + - * / %

Unary and Binary operators

Unary operator	Binary operator
1 operator 1 operand E.g. -4	1 operator 2 operand E.g. 2+4

if else- switch case

If else	Switch case
All the different data types can be used	Only char and int type can be used
Does not have a default operation	Has a default operation
Can be tested for logical and relational fn	Only for equality

for and while

For	While
Can be used for only fixed iteration	Can be used when the no of times for a loop to be done is not known
Can use only char or int type	Any data type

do-while and while statement

Do while	While
Exit controlled	Entry controlled
If the condition is false then also the loop will be done once	If the condition is false then the loop will not be done at all
E.g.: int a=1; While (a>=5) { a++; } loop will not be done even once	int a=1; do { a++; }while(a>=5); loop will be done at least once

/ and %

/	%
Returns the quotient	Returns the remainder
E.g.: int c=7/2 answer =3 float c=7/2 answer = 3.5	E.g.: int c=7%2 Answer =1

Compiler and interpreter

Compiler	Interpreter
It is a software which converts high level language into machine language as a whole	It is a software which converts high level language into machine language line by line
It is faster	Slower
Debugging is harder	Debugging is easier

character and string literal

Character literal	String literal
A single character enclosed within single quotes	Zero or more characters enclosed within double quotation.

primitive and user defined datatype

Primitive data type	User defined data type
These are built in data types	Created by user
The sizes of these objects are fixed	The sizes of these data types depend upon their constituent members
These data types are available in all parts of a java program	The availability of these data types depends upon their scope

print() and println()

Print()	Println()
The successive output will come in the same line	The successive output will come in a new line

Break & continue

Break	Continue
it serves as a terminator from the enclosed loop,(i.e.) it comes out of an inner loop. And transfers the control to the statement after the loop.	Continue-causes the control to transfer from within the block to the next iteration of the loop.
Break -E.g. While(amount<100) { if(amount == 11) break; } if amount is 11 then the control will be transferred to the statement following the while loop.	E.g. for(i=0;i<5;i++) { if(a==0) continue; System.out.println(a*a);

Operator Precedence

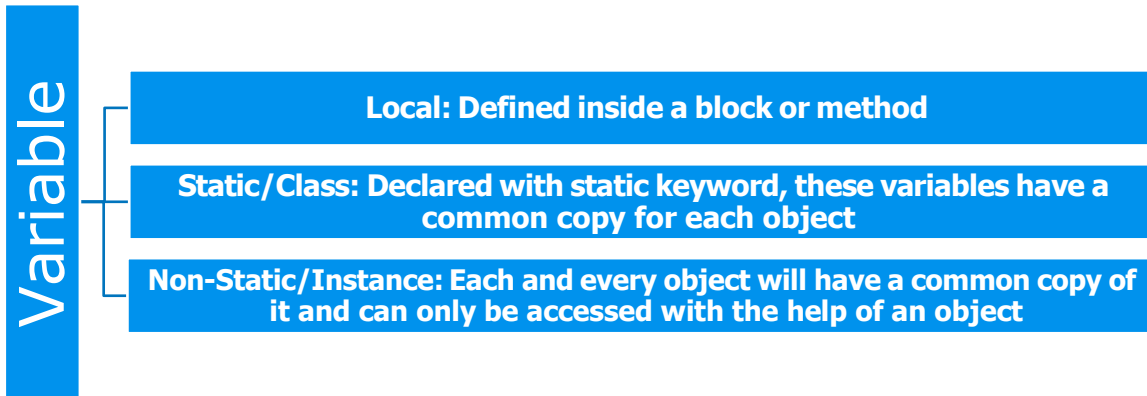
Precedence	Operator	Type	Associativity
15	() [] .	Parentheses Array subscript Member selection	Left to Right
14	++ --	Unary post-increment Unary post-decrement	Right to left
13	++ -- + - ! ~ (type)	Unary pre-increment Unary pre-decrement Unary plus Unary minus Unary logical negation Unary bitwise complement Unary type cast	Right to left
12	* / %	Multiplication Division Modulus	Left to right
11	+ -	Addition Subtraction	Left to right
10	<< >> >>>	Bitwise left shift Bitwise right shift with sign extension Bitwise right shift with zero extension	Left to right
9	< <= > >= instanceof	Relational less than Relational less than or equal Relational greater than Relational greater than or equal Type comparison (objects only)	Left to right
8	== !=	Relational is equal to Relational is not equal to	Left to right
7	&	Bitwise AND	Left to right
6	^	Bitwise exclusive OR	Left to right
5		Bitwise inclusive OR	Left to right
4	&&	Logical AND	Left to right
3		Logical OR	Left to right
2	? :	Ternary conditional	Right to left
1	= += -= *= /= %=	Assignment Addition assignment Subtraction assignment Multiplication assignment Division assignment Modulus assignment	

Objects & Classes

Object An identifiable entity with certain characteristics, behavior, and attributes.

Classes A group or collection of similar objects with similar characteristics and behavior. (An instance of a class).

Variable A named memory location to store values.



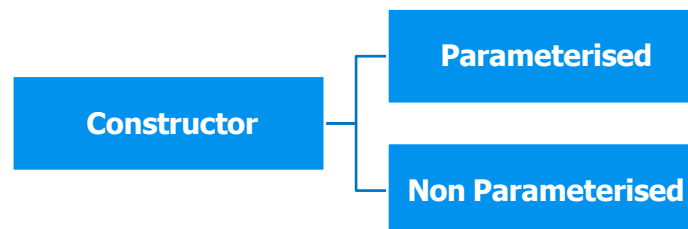
Constructor

Constructor is a special method which is used to initialize instance/non-static variable.

Properties of constructors

- A constructor has the same name as the class
- It does not have a return type, not even void
- It doesn't have a calling statement
- The constructor gets called when an object is created

Types of constructors



Examples:

Parameterized:

```
1. Int num1, num2, sum;  
2. Sum(int num1, num2){  
3.   num1=num1;  
4.   num2=num2;  
5. }
```

Non-Parameterized:

```
1. Int num1, num2, sum;  
2. Sum(){  
3.   Sum=0;  
4. }
```

Constructor Overloading When there are more than one constructor in a program, it's called constructor overloading.

“this” keyword “this” is a keyword which refers to the current object.

Example:

```
1. String flavor; float cost;
2. Fruitjuice(String flavor, float cost){
3.     this.flavor=flavor;
4.     this.cost=cost;
5. }
6.
```

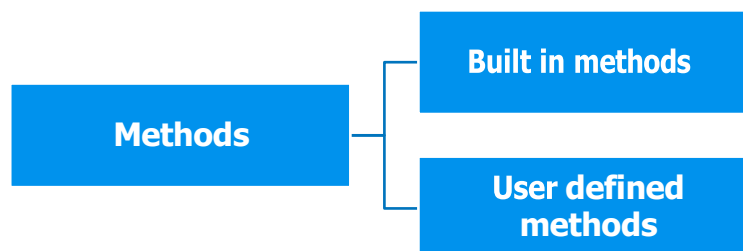
Return types indicate the type of outcome of a method to be returned to its caller. Ex: int, double, float, char, short, byte, Boolean, void

Example:

```
1. Public int sum(int a, int b){
2.     c=a+b;
3.     Return c;
4. }
```

Function/Method

Function is a set of Java executable statements enclosed in a function definition.



Advantages of methods:

- Reusability
- Manage complex programs into smaller parts
- Hide details

Method definition/prototype refers to the first line of a method which contains the access specifier, modifier, return type, method name and the method signature.

Syntax

```
1. <access specifier> <modifier> <return type> <method name> (list of
   parameters)
2. {
3.   <statements>
4. }
5. //eg:
6. public static int(int a, int b){
7.   int c=a+b;
8.   return c;
9. }
```

Method call syntax:

```
1. <object name>.<method name>(arguments);
2. //Example:
3. a.example();
```

new keyword is used to dynamically allocate memory for the object

Static keyword is a modifier which identifies the method as a class method. It means that the method is not called through an object

Access-specifier: Keywords (except friendly) that control the visibility of a data member

Private: the data members or member methods which are specified as private can be used only within the scope of the class. These members are not accessed outside the class.

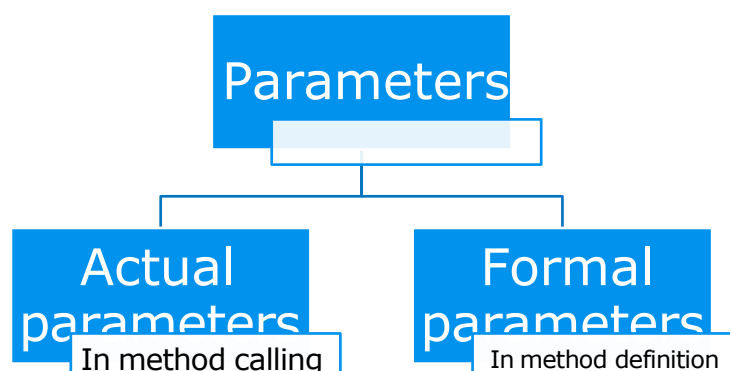
Public: The class members specified as public can be used even outside the visibility of a class

Protected: the protected members are used in the class as private members which can only be applied within the class but can be inherited to another class

Return types indicate the type of outcome of a method to be returned to its caller. Ex: int, double, float, char, short, byte, Boolean, void

Method signature: Collection of data type and variable names written inside a function definition.

Parameters: The value which is passed into the function to instantiate



Actual parameters: The parameters that appear in method calling

Formal parameters: The parameters that appear in method definition

Actual Parameters	Formal Parameters
Appear in method calling	Appear in method definition
Original value	Copied value of actual parameters

**Ways of
passing
values to a
function**

Pass/call by value:

Any change made in the formal parameters will not reflect in the actual parameters

Pass/call by reference:

Any change made in the formal parameters will reflect in the actual parameters

Call by value	Call by reference
Changes made in the formal parameters will not reflect in the actual parameters	Changes made in the formal parameters will reflect in the actual parameters
Primitive type data are passed to the method using pass by value	Reference type data are passed to the method using pass by value
It is a pure function	It is an impure function

Pure function/accessor method: A method that returns a value but does not change the state of an object

Impure function/mutator: A method that may not return a value but change the state of an object

Recursive Function: A function that calls itself / A function that refers to itself for execution

Method Overloading

Method overloading: Multiple functions sharing the same name with different parameters/ method signature.

Example:

```
1. void Area(int side){
2. area=side*side;
3. }
4. void Area(int l, int b){
5. area=l*b; }
```

Arrays

Array: An Array is a set of like variables which are referred to by a common name. A number called subscript/index is used to distinguish one from the other.

Syntax

```
1. <data type> <array name>[]=new <data type>[<n>];
```

‘n’ denotes the maximum number of elements that the array can hold.

Assigning values for an Array

Ex:

```
1. int arr[]={1, 2, 3, 4}
```

.length function – Tells the number of elements in an array Ex:

```
1. int len=arr.length;
```

Linear search: The search element is checked with all the elements of the array. If the element is found, the flag variable is made to ‘1’ otherwise, it remains ‘0’.

Linear Search	Binary Search
Can work with both sorted and unsorted arrays.	Works only with sorted arrays.
Reads and compares each element of the array one by one.	Works by dividing the array in two segments out of which only one needs to be searched.

String Handling & Library Classes

In Java, String is an object which contains a sequence of characters. String class is used to create and manipulate strings. The String class is available in java.lang package.

Declaration and Assigning a String

```
1. //Declaration:
2. String <variable>;
3. //Ex:
4. String str;
5. //Assigning:
6. <variable>=<String literal>;
7. //Ex:
8. str="Hello World!";
```

Input a String

```
1. //For a string without any space (For a single word):
2. <variable>=<Scanner object>.next();
3. Str=sc.next();
4. //For a String with spaces (For a sentence):
5. <variable>=<Scanner object>.nextLine();
6. Str=sc.nextLine();
```

String Functions

For all the below examples, str="COMPUTER"; **Output** will be displayed as a **single line comment** (//).

Quick Tip

1. **Function names start with lowercase and then the second word starts with uppercase letters.** Eg: **i**ndex**O**f();
2. **Topics asked in board questions are marked with** 

.length() (int)

This function is used to return the **length** of the string.

Syntax with example:

```
1. <int variable>=<string var>.length();
2. int Len=str.length();
3. //8
```

.charAt() (*char*)

This function returns the **character** from the given index.

Syntax with example:

```
1. <char variable>=<string var>.charAt(<index>);  
2. char ch=str.charAt(2);  
3. //0
```

.indexOf() (*int*)

This function returns the **index of first occurrence** of a character.

Syntax with example:

```
1. <int variable>=<string var>.indexOf(<character>);  
2. int idx=str.indexOf('M');  
3. //2
```

.indexOf(char ch, int start_index) (*int*)

This function returns the **index** of a given **character** from the given **index**.

Syntax with example:

```
1. <int var>=<String var>.indexOf(<char var>,<int var>);  
2. char ch='M';  
3. int ind=str.indexOf(ch, 1);  
4. //2
```

.lastIndexOf(char ch) (*int*)

This function returns the **index** of the **last occurrence** of a given character.

Syntax with example:

```
1. <int var>=<String var>.lastIndexOf(char ch);  
2. int ind=str.lastIndexOf('E');  
3. //6
```

.substring(int start_index, int last_index) (*String*)

This function is used to extract a **set of characters** simultaneously from a given index upto the end of the String or till a given index.

Syntax with example:

```
1. <String var>=<String var>.substring(<int var>,<int var>);  
2. String ext=str.substring(3);  
3. //PUTER
```

.toLowerCase() (*String*)

This function is used to convert a given String to **lowercase** letters (entire string).

Syntax with example:

```
1. <String var>=<String var>.toLowerCase();
2. String lc=str.toLowerCase();
3. //computer
```

.toUpperCase() (String)

This function is used to convert a given String to **uppercase** letters (entire string).

Syntax with example:

```
1. <String var>=<String var>.toUpperCase();
2. String uc=str.toUpperCase(ind);
3. //COMPUTER
```

.replace(char old, char new) (String)

This function is used to **replace** a **character or a sequence of characters** in a String with a new character or sequence of characters. (**NOTE:** This does not work with int values)

Syntax with example:

```
1. <String var>=<String var>.replace(<char var>,<char var>);
2. String rep=str.replace("PUTER","PUTE");
3. //COMPUTE
```

.concat(String second) (String)

This function is used to **concatenate/join** two Strings together. (**NOTE:** This does not add any spaces in-between)

Syntax with example:

```
1. <String var>=<String var>.concat(s);
2. String s="STUDENT";
3. String con=str.(s);
4. //COMPUTERSTUDENT
```

.equals(String str) (boolean)

This function is used to check for **equality** between two Strings. (**NOTE:** This function returns a **boolean** value. This function cannot be used for characters. //You can simply use == for characters. This can be used in if statements)

Syntax with example:

```
1. <boolean var>=<String var>.equals(<String var>);
2. String s="COMPUTER";
3. boolean chk=str.equals(s);
4. //true
```

.equalsIgnoreCase(String str) (boolean)

This function does the same function of .equals() function.

The only difference is that it does not care about the case (It **ignores the case**).

Syntax with example:

```
1. <boolean var>=<String var>.equalsIgnoreCase(<String var>);
2. boolean chk=str.equalsIgnoreCase("cOmPuTeR"); //true
```

.compareTo(String str) (int)

This function is used to **compare** two Strings. It also checks whether a String is **bigger or smaller** than the other and returns a suitable **int value**. It returns **0** if both are **equal**. A **positive** value when the **first is bigger** than the second and a **negative** value when the **second String is bigger** than the first. It returns the **no. of additional characters** when both the Strings' **first sequence of characters are equal** but the other has additional characters.

Syntax with example:

```
1. <int var>=<String var>.compareTo(<String var>);
2. String s="SCIENCE";
3. int cmp=str.compareTo(s);
4. //A, B, C, (C is the 3rd letter in the Alphabet and S is the 19th)
5. //the value of cmp will be -16 because (3-19=-16)
```

compareToIgnoreCase(String str) (int)

This function does the same function as .compareTo but it **ignores the case**.

Syntax with example:

```
1. <int var>=<String var>.compareToIgnoreCase(<String var>);
2. int cmp=str.compareToIgnoreCase("cOmPuTeR");
3. //0
```

.trim() (String)

This function removes **spaces** at the **start and end** of the String. (**NOTE:** This function does not remove spaces in-between characters)

Syntax with example:

```
1. <String var>=<String var>.trim();
2. Str="    He llo World! ";
3. String trm=str.trim();
4. //He llo World!
```

.startsWith(String str) (boolean)

This function is used to check if the given String is a **prefix** to the other.

Syntax with example:

```
1. <boolean var>=<String var>.startsWith(<String var>);
2. pfx="COM"
3. boolean chk=str.startsWith(pfx);
4. //true
```

.endsWith(String str) (boolean)

This function is used to check if a given String has a specified **suffix**.

Syntax with example:

```
1. <boolean var>=<String var>.ends with(<String var>);
2. boolean chk=str.endsWith("TER");
```

.equals()	.compareTo()
Returns a Boolean value	Returns a an int value
It checks for equality between two Strings	It checks if a String is equal, bigger or smaller than the other.

Difference Between equals() and compareTo() functions 

Library Classes & Wrapper Classes

For better understanding:

Before we get into Library Classes & Wrapper Classes, it's important to know what is a primitive and composite data types.

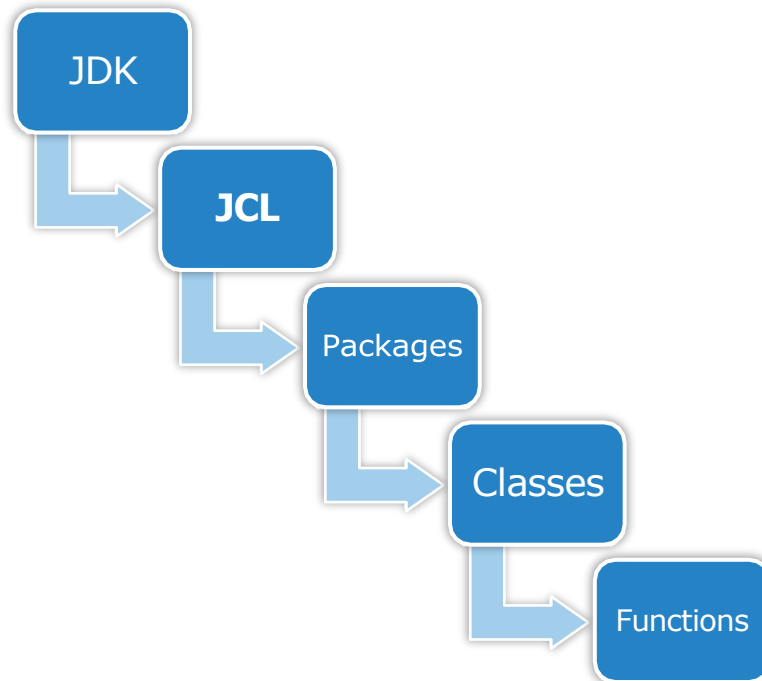
Primitive Data Type: These are **fundamental built-in data types** of **fixed sizes**. **Ex:** int, long, float

 **Composite/Reference/User-Defined Data Type:** These are data types **created by the user**. The availability of these data types depends upon their **scope and sizes** depend upon their **constituent members**.
Ex: array, class, object

Primitive data type	Composite data type
These are built in data types	Created by user
The sizes of these objects are fixed	The sizes of these data types depend upon their constituent members
These data types are available in all parts of a java program	The availability of these data types depends upon their scope

Library Classes !

JDK (Java Development Kit) V1.5 and above contains Java Class Library (JCL) which contains various packages. Each package contains various classes containing different built-in functions.



Ex: *java.lang, java.math*

Wrapper Class

Wrapper Classes are a part of java.lang (A Library Class Package). Wrapper classes **contain primitive data values in terms of objects/ Wrapper Class wraps a primitive data type to an object**. There are 8 wrapper classes in Java. **Ex:** Integer, Byte, Double

(NOTE: Wrapper Classes always start with an uppercase letter

Ex: Integer, Boolean, Float)

Need for Wrapper Classes

- To store primitive values in the objects
- To convert a string data into other primitive types and vice- versa

Wrapper Class	Primitive Type
Byte	Byte
Short	Short
Integer	Int
Long	Long
Float	Float
Double	Double
Character	Char
Boolean	boolean

Functions/Methods of Wrapper Classes

Conversion from String to Primitive types

For converting String to any primitive data type, Wrapper Class functions can be used. For any primitive data Wrapper Class, the **parse**<prm data type>(<String arg>) (or) **valueOf**(<String arg>) functions can be used.

Eg: `int i=Integer.parseInt(s); int j=Integer.valueOf(s);`

For better understanding:

```
1. <prm data type var>=<prm data type Wrapper Class>.parse<prm data
   type name>(<String arg>);
2. <prm data type var>=<prm data type wrapper class>.valueOf(<String
   arg>);
3.
4. //Examples:
5. int a=Integer.parseInt("238");
6. double b=Double.parseDouble("23.45");
7. int c=Integer.valueOf("37");
8. float d=Float.valueOf("42.87");
```

Examples of each <> (In the above syntax):

prm data type: int a | double b

prm data wrapper class: Integer | Double

prm data type name: Int | Long | Double

String arg: "38.743" | "1874293856"

Conversion from primitive type data to String

For converting a primitive type data to a String, the **toString()** Wrapper Class function can be used.

Ex: Integer.toString() | Double.toString()

```
1. <String var>=<Wrapper Class>.toString(<prm data arg>);  
2. String cnv=Integer.toString(38);  
3. String dbl=Double.toString(94.53);
```

Boxing, Unboxing & Autoboxing

Boxing

Conversion of primitive type data **to an object**.

Syntax with example:

```
1. <wrapper class> <object name>=new <wrapper class>(<prm type arg>);  
2. int a=239;  
3. Integer x=new Integer(a);
```

Unboxing

Conversion of an object **to primitive type data**.

Syntax with example:

```
1. <int var>=<wrapper class obj>  
2. int b=x;
```

Autoboxing

Boxing is the mechanism and autoboxing is the feature of the compiler which generates the boxing code.

Syntax with example:

```
1. <wrapper class> <object name>=new <wrapper class>(<prm type arg>);  
2. int a=239;  
3. Integer x=new Integer(a);
```

Character

Character is defined as a letter, a digit or any special symbol/UNICODE enclosed within single quotes. **Ex:** '@', 's', '5'

Assigning a character

A Character is declared under char data type.

Syntax with example:

```
1. char <var name>='<char literal>';  
2. char ch='a';
```

Input a character

A Character is declared under char data type.

Syntax with example:

```
1. <char var>=<Scanner obj>.next().charAt(0);  
2. ch=sc.next().charAt(0);
```

Character Functions

Character.isLetter() (boolean)

This function is used to check if a given argument is a letter or not.

Syntax with example:

```
1. <boolean var>=Character.isLetter(<char arg>);  
2. boolean chk=Character.is('A'); //true
```

Character.isDigit() (boolean)

This function is used to check if a given argument is a digit or not.

Syntax with example:

```
1. <boolean var>=Character.isDigit(<char arg>);  
2. boolean chk=Character.is('7'); //true
```

Character.isLetterOrDigit() (boolean)

This function is used to check if a given argument is either a letter or a digit or none of these.

Syntax with example:

```
1. <boolean var>=Character.is(<char arg>);  
2. boolean chk=Character.is('A'); //true
```

Character.isWhitespace() (boolean)

This function is used to check if a given argument is a blank/gap/space or not.

Syntax with example:

```
1. <boolean var>=Character.is(<char arg>);  
2. boolean chk=Character.is('A'); //false
```

Character.isUpperCase() (boolean)

This function is used to check if a given argument is an uppercase letter or not.

Syntax with example:

```
1. <boolean var>=Character.is(<char arg>);  
2. boolean chk=Character.is('A'); //true
```

Character.isLowerCase() (boolean)

This function is used to check if a given argument is a or not.

Syntax with example:

```
1. <boolean var>=Character.is(<char arg>);  
2. boolean chk=Character.is('A'); //false
```

Character.toUpperCase() (char)

This function is used to convert/returns a given argument/character/letter to/in uppercase character/letter.

Syntax with example:

```
1. <char var>=Character.toUpperCase(<char arg>);  
2. char uc=Character.toUpperCase('a'); //A
```

Character.toLowerCase() (char)

This function is used to convert/returns a given argument/character/letter to/in lowercase character/letter.

Syntax with example:

```
1. <char var>=Character.toLowerCase(<char arg>);  
2. char lc=Character.toLowerCase('A'); //a
```


Differentiate between the following

Unary & Binary operator (ICSE 2019)

Unary	Binary
Works on single operand	Works on 2 operands
E.g., ++, --	E.g., +, -, *, /

If else if and switch case (ICSE 2019 Marking Scheme)

If else if	Switch case
Range if values are checked	Only one value is compared
Supports both primitive and composite data type	Supports only integer, char
Relational/Logical expression can be checked	Only equality can be checked

Linear search and Binary search (ICSE 2019 Marking Scheme)

Linear Search	Binary Search
Array need not be sorted	Array must be sorted
Checks each item	Does not check each item
Checks the search value from the 0 th index	Checks the search value from the middle value of the array
It checks in a sequential order	It checks by dividing the array into two halves

Call by value and Call by reference (ICSE 2019 Marking Scheme)

Call by value	Call by reference
It works with primitive data type	It works with reference data type
The original value of variable remains unchanged	The original value of variable changes
Operation is performed on duplicate value of variables	Operation is performed on original values of variables.
It is also called as pure function.	It is also called as Impure function.

Searching and sorting (ICSE 2018 Marking Scheme)

Searching	Sorting
To find an element in an array is called searching	To arrange the elements in an order is called sorting
Eg: Linear Search	Eg: Bubble sort

isUpperCase and toUpperCase (ICSE 2018 Marking Scheme)

isUpperCase	toUpperCase
CHECKS whether a given character is an uppercase letter or not	CONVERTS the character to its upper case
Output of this function is boolean	Output of this function is character

While loop and do while loop (ICSE 2018 Marking Scheme)

While loop	Do while loop
Entry controlled loop	Exit controlled loop
If condition is false in the beginning, loop never executes	Loop will be execute at least once even if the condition is false
Minimum reputation is 0	Minimum reputation is 1

Constructor and function (ICSE 2017)

Constructor	Function
The constructor has the same name as the class	A function has a different name than that of the class
It does not have a return type, not even void	It must have a return type

Break and Continue

Break	Continue
Used to terminate a block in which it exists	Used to transfer the control to the next iteration
Break statement can be used in switch case as well as loop statements	It cannot be used in a switch case

Unboxing and Boxing (ICSE Prelim)

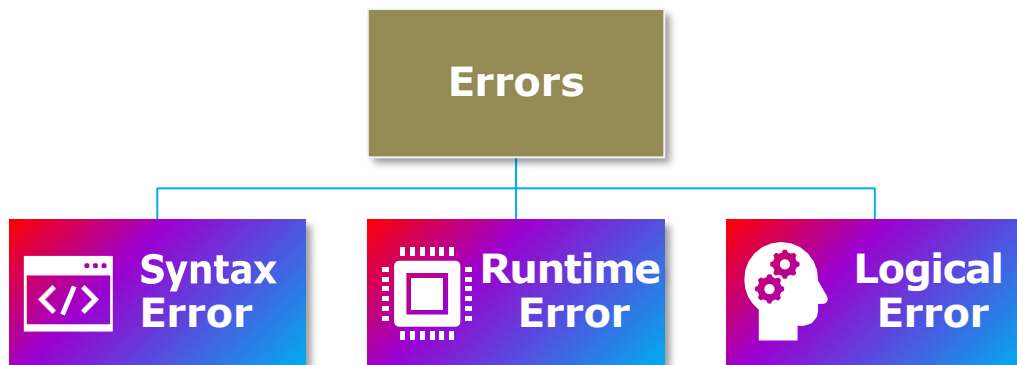
Unboxing	Boxing
Conversion of an object to primitive data type	Conversion of primitive type data to an object
Passed as an argument to a function which is expecting a primitive data type variable.	Passed as an argument to a function which is expecting a wrapper class object.

Private and public access specifier

Private	Public
These members are accessible only within the same class where it's declared.	These members are accessible within the package as well as outside the package.
Higher level of encapsulation	Lowest level of encapsulation

Note: All classes in the package have access to the member defined friendly. The classes outside the package doesn't have access to the member.

! ERRORS IN JAVA



Types of Errors (Icons are for illustration purposes only. FrozenNotes cannot be held liable for incorrect representation)

Details and Examples:

1. Syntax Errors (A.K.A Compile Time Error) `</>`

Errors which prevent a code from execution due to **incorrect syntax** are called Compile Time Errors. These may include mistakes like missing semicolon, missing brackets, grammatical mistakes and more...

Example:

```
1. System.out.print("Hello") //Missing semicolon
2. public stat void hello(){System.out.println("Hello World!");} //Missing brackets & grammatical
3. int a=7, b=5;
4. z=a+b; //uninitialized variable z
```

2. Runtime Errors

Errors which are **detected** during the **execution** of the program. Runtime errors occur when a program does not contain any syntax errors but asks the computer to do something that the computer is **unable to reliably do**.

```
1. int a=248, b=0, c=0;
2. c=a/b; //Division by 0 is not possible
```

3. Logical Errors

Error occurred when the program **does not perform** the **expected** function and yields a different result.

These programs get **successfully compiled** and executed but gives a **different** result than what is expected.

```
1. int a=34, b=7, c=0; c=a/b; //Incorrect operator
2. System.out.println("The reminder of a divided by b is "+c+" and I'm
confident about it");
```

Infinite Loop

A sequence of statements which **iterates indefinitely** until the system crashes or the execution is terminated. These are encountered/occur when the given **terminating condition** is not **satisfied**.

Example:

```
1. for(i=0; i<1; i--){
2.     System.out.println("Ah! I won't stop, do whatever you can but please don't
    terminate!");
3. }
```

Difference between == and equals()

==	equals()
== is a relational operator	equals() is a String class function
== can be used for all type of data except Strings	Equals() can only be used for Strings

Suggestions For Candidates

From The CISCE Council

- Follow the concepts given in the **scope of the syllabus**.
- **Clarify** the concepts and practise them both, on the **paper** and on the **computer**.
- **Solve** a lot of **problems** based on all the concepts.
- Learn the **syntax** and **working** of every **concept** properly, with suitable examples.
- **Comprehend** the **key terms/definitions** and then learn.
- Practise **Library class** and its various **functions**. Check its **output** on the **computer** to understand their working.
- **Develop** the habit to do **dry run of a program**, which you write. This will help in better understanding of concepts and aid in solving questions.
- **Apply simple logic** in programs to get desired output.
- Complete the **assignments** and **cross check** on the **computer** for their proper working. Get the assignments **checked** by your **teacher**.
- Solve **previous years' ICSE question papers** to understand the types of questions asked and how to attempt an ICSE question paper.
- Check **output-based questions** on the **computer**.
- Design your **own questions** for **string functions, math functions, loops - for, while, do while, etc.**
- Check for the **logic** for different variety of **numbers-based questions** and develop a **logic** for the same.
- Do not **resort** to **rote learning** this subject but **understand** and **practise** the concepts learnt **regularly**.
- Follow a proper **study schedule** when preparing for the examination.
- **Revise** and integrate the concepts studied in **Class IX** with the **Class X** syllabus.
- **Avoid selective** study.
- Give **equal importance** to **all** the **topics** mentioned in the **syllabus**.
- **Practise** each topic/sub –topic with as many examples as possible.
- Being an application-oriented subject, **apply** what is **taught** in the program and **explain** its outcome.
- Give **sufficient practice** to **output-based** questions.

- Learn **correct** use of **all statements** to **eliminate syntax errors**.
- **Practise** programs on various types of loops, their working, **conversion** from one **loop to another** and working of **nested loop**.
- Use **proper variable names** and ensure that every program has a **variable description table**.
- **Avoid** writing **abbreviations** like SOP, SOPLN, PSVM.
- Understand the **logic** of a program **instead** of **memorising** it.
- **Explain** programs using **Mnemonic variables** and **comments**.
- After writing the program, **dry run** it with **different inputs**
- Write **variable description** /Mnemonic codes for **every program**.
- **Read** the **questions carefully** and write the answers **according** to their **requirements**.
- **Utilize** the **reading time** to clearly **understand** the **nature** of the **question**.
- **Do not waste time** in writing **long variable descriptions**
- At least write **2 comments** in a program
- Give **proper spacing** in-between **blocks** in a program
- **Split** the program **logic** into **different parts** for better analyzation
- **Do not skip** any **questions**, try it. At the same time **avoid writing alarmingly incorrect answers**, writing these type of answers may irritate the examiner.
- **Do not CHEAT/UNFAIR MEANS.**